

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

I M. Varshith Reddy(192524174) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M. Varshith Reddy

Annexure A

Scenario-based

1. Secure Data Transmission using Block Cipher

A company is transmitting repeated financial data blocks over a network. Initially, they used ECB mode and noticed patterns in ciphertext.

- A. Explain why this issue occurs.
- B. Suggest a more secure block cipher mode and justify your choice based on the scenario.

2. Choosing an Encryption Algorithm for Banking System

A banking application needs high security and moderate performance for encrypting customer transactions. The available options are DES, 3-DES, and Blowfish.

- A. Identify the most suitable algorithm for this scenario.
- B. Justify your choice based on security strength and efficiency.

3. Key Exchange in an Unsecured Network

Two users want to establish a secure communication channel over a public network using Diffie-Hellman key exchange. However, the network is vulnerable to attacks.

- A. Explain how the key exchange works in this scenario.
- B. Identify a possible security threat and suggest a solution to mitigate it.

4. Public Key Cryptography for Email Security

An organization plans to implement RSA for secure email communication.

- A. Describe how RSA can be used for encryption and decryption in this scenario.
- B. Explain how key distribution and management play a role in maintaining security.

5. Cryptography for IoT Devices

A smart home system uses low-power IoT devices that need secure communication. The designer must choose between RSA and Elliptic Curve Cryptography (ECC).

- A. Analyze the constraints of IoT devices.
- B. Recommend a suitable cryptographic approach and justify your answer.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes wellreasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

CO2 Assessment – Cryptography and Network Security (CSA51)

Scenario-Based Answers (Detailed)

1. Secure Data Transmission using Block Cipher

A. Why does the ECB pattern issue occur?

ECB (Electronic Codebook) mode is the simplest mode of operation for a block cipher. In ECB, the plaintext is divided into fixed-size blocks (for example, 64 bits for DES or 128 bits for AES), and each block is encrypted independently using the same key, with no interaction between blocks and no use of an initialization vector (IV).

This independence is precisely the source of the problem. Because the encryption of a block depends only on the plaintext of that block and the key – and not on any other block, its position, or any random value – the transformation is deterministic: identical plaintext blocks will always produce identical ciphertext blocks, every single time, as long as the key stays the same.

In the described scenario, the company is transmitting repeated financial data blocks – this could be recurring transaction headers, repeated account number formats, standard message templates, or padded/zero-filled fields that occur frequently in structured financial records. When these repeated plaintext patterns are encrypted under ECB, they generate repeated ciphertext blocks. An attacker who is only observing the ciphertext (without breaking the key) can still detect these repeating patterns and infer:

- That certain transactions or fields are identical or similar in structure.
- Statistical relationships between different messages (e.g., which customers send similar transaction types).
- Potential replay opportunities – since a block encrypted once will always look the same, an attacker could copy and resend a

previously observed ciphertext block into a new context (a block replay/cut-and-paste attack), even without knowing the plaintext or key.

A classic illustration of this weakness is encrypting an image with ECB — the outline and shapes of the original image remain visible in the ciphertext because uniform regions of pixels (repeated data) map to uniform ciphertext blocks. The same conceptual leakage applies to financial data: even though the attacker cannot read the actual numbers, the existence of a pattern is itself a security leak, violating the fundamental cryptographic principle that ciphertext should be indistinguishable from random noise (semantic security).

B. A more secure block cipher mode, and justification

Recommended mode: CBC (Cipher Block Chaining), or preferably GCM (Galois/Counter Mode) for this specific financial scenario.

How CBC solves the problem: CBC introduces dependency between blocks. Before encrypting a plaintext block, it is first XORed with the ciphertext of the *previous* block:

$$C[i] = E(K, P[i] \oplus C[i-1])$$

For the very first block, a random, unique Initialization Vector (IV) is XORed with the plaintext before encryption. Because each block's ciphertext depends on all previous blocks (through the chaining) and on a fresh random IV per message, two identical plaintext blocks will now produce completely different ciphertext, as long as they don't happen to occur in messages with the same IV and identical preceding blocks. This eliminates the pattern-leakage problem seen in ECB.

Why GCM is even better suited for a banking/financial network: While CBC solves the pattern-leakage issue, it does not provide data integrity or authentication on its own — an attacker could still tamper with ciphertext blocks (bit-flipping attacks) without detection, which is a serious concern for financial transactions where a single altered digit could change a transaction amount. GCM combines:

1. Confidentiality — using CTR-mode-style encryption, which (like CBC)

avoids repeating patterns because each block is combined with a unique counter value.

2. Authentication — via a Galois-field-based MAC (authentication tag) that is verified on decryption. If even a single bit of the ciphertext is altered in transit, decryption will fail integrity verification.
3. Performance — GCM's counter-based structure allows blocks to be encrypted/decrypted in parallel, which is faster than CBC (a strictly sequential mode) on modern multi-core systems — important for high-throughput banking networks with heavy transaction volume.

Conclusion: For a company repeatedly transmitting structured, sensitive financial data, moving away from ECB to CBC (at minimum) removes pattern leakage, but adopting AES-GCM is the stronger recommendation because it simultaneously solves the confidentiality issue *and* protects against unauthorized tampering — both essential for financial-grade security and typically mandated by compliance frameworks like PCI-DSS.

2. Choosing an Encryption Algorithm for a Banking System

A. Most suitable algorithm

Among the three given options — DES, 3-DES, and Blowfish — the most suitable choice for a banking application requiring high security with moderate performance is 3-DES (Triple DES).

(Note: In real-world modern banking deployments, AES would typically be the first choice since it is faster and more secure than all three options listed here. But since the question restricts the choice to DES, 3-DES, and Blowfish, 3-DES is the correct answer among those three.)

B. Justification based on security strength and efficiency

Why not DES: DES uses a fixed 56-bit key. With modern computational power (including cloud-based brute-force and specialized hardware like FPGAs/ASICs), a 56-bit keyspace can be exhaustively searched in a matter of hours to days. DES was formally broken via brute force decades ago (e.g., the EFF's "Deep Crack" machine in 1998) and is now

considered cryptographically obsolete. Using DES for banking transactions today would expose customer financial data to trivial brute-force attacks — completely unacceptable for a system requiring high security.

Why 3-DES is the appropriate middle ground: 3-DES strengthens the original DES algorithm by applying it three times in sequence, typically using either two or three distinct keys, in an Encrypt-Decrypt-Encrypt (EDE) construction:

$$C = E(K_3, D(K_2, E(K_1, P)))$$

This construction effectively raises the security strength to approximately 112 bits (with two-key 3-DES) or up to 168 bits (with three-key 3-DES, though meet-in-the-middle attacks reduce the *effective* strength to roughly 112 bits even in the three-key case). This is a massive improvement over plain DES's 56-bit strength and, while not as fast or as strong as AES, it remains resistant to all currently known practical brute-force and cryptanalytic attacks when used correctly.

Additionally, 3-DES has a long history of formal validation:

- It has been FIPS-approved and widely adopted in legacy financial systems, including older EMV chip card standards and PIN-block encryption in ATM networks.
- Its widespread historical adoption in the banking sector means many existing systems, hardware security modules (HSMs), and payment terminals already support it natively, easing integration.

Why not Blowfish (in this scenario): Blowfish is a fast, license-free symmetric cipher designed by Bruce Schneier, and it can offer strong security with key sizes up to 448 bits. However, it has two notable drawbacks relevant to a banking context:

1. 64-bit block size — like DES, Blowfish uses a small 64-bit block. When large volumes of data are encrypted under the same key (common in high-transaction-volume banking), the probability of a "birthday bound" collision (two identical ciphertext blocks appearing by chance) increases significantly after roughly 2^{32} blocks are encrypted — a real risk in high-throughput financial

networks (this is the same class of vulnerability behind the "SWEET32" attack demonstrated against 64-bit block ciphers).

2. Lack of formal standardization/certification — unlike 3-DES, Blowfish was never adopted as a national or international standard (no FIPS validation), which matters significantly in regulated banking environments where compliance auditors expect validated, standardized cryptographic algorithms.

Conclusion: Balancing the requirement for high security against moderate performance, 3-DES offers a proven, standardized, and significantly more secure alternative to DES, while avoiding the block-size and certification concerns associated with Blowfish — making it the most suitable choice among the three given algorithms for this banking scenario.

3. Key Exchange in an Unsecured Network

A. How the Diffie-Hellman key exchange works in this scenario

The Diffie-Hellman (DH) protocol allows two parties who have never communicated before to establish a shared secret key over a public, potentially insecure channel, without ever transmitting the secret key itself. It relies on the mathematical difficulty of the discrete logarithm problem — given g , p , and $g^x \bmod p$, it is computationally infeasible to recover x for sufficiently large p .

Step-by-step process for the two users (say, User A and User B):

1. Public parameter agreement: Both parties publicly agree on two values: a large prime number p and a primitive root (generator) g modulo p . These values do not need to be secret — they can be sent openly over the network.
2. Private key selection:
 - User A randomly selects a private integer a (kept secret).
 - User B randomly selects a private integer b (kept secret).
3. Public key computation and exchange:
 - User A computes $A = g^a \bmod p$ and sends A to User B over the public network.
 - User B computes $B = g^b \bmod p$ and sends B to User A over the public

network.

- Note that only A , B , g , and p travel over the network — the private values a and b are never transmitted.
4. Shared secret computation:
- User A computes the shared key: $K = B^a \bmod p = (g^b)^a \bmod p = g^{(ab)} \bmod p$
 - User B computes the shared key: $K = A^b \bmod p = (g^a)^b \bmod p = g^{(ab)} \bmod p$
 - Both arrive at the identical shared secret K , even though they never directly exchanged it.
5. This shared secret K can then be used as (or to derive) a symmetric session key for encrypting the actual communication (e.g., using AES) between the two users.

Because an eavesdropper observing the network only sees p , g , A , and B — but not a or b — and because computing a or b from these public values (the discrete logarithm problem) is computationally infeasible for large enough p , the shared secret remains protected against passive eavesdropping.

B. Security threat and mitigation

Threat: Man-in-the-Middle (MITM) Attack

The critical weakness of the plain Diffie-Hellman protocol, as described, is that it provides no authentication of the two communicating parties. Neither User A nor User B has any cryptographic proof of who is actually on the other end of the exchange — they simply trust that the public value they received (A or B) genuinely came from the other legitimate user.

This opens the door to a Man-in-the-Middle attack:

1. An attacker (say, Eve) intercepts the communication channel between User A and User B.
2. When User A sends her public value A intended for User B, Eve intercepts it and instead sends her own public value (A_e) to User B, pretending to be User A.
3. Similarly, Eve intercepts User B's public value B and sends her own public value (B_e) to User A, pretending to be User B.

4. As a result, User A establishes a shared key with Eve (believing it's shared with User B), and User B independently establishes a *different* shared key with Eve (believing it's shared with User A).
5. Eve can now decrypt, read, modify, and re-encrypt every message passed between A and B, relaying the (possibly altered) traffic between them — while both A and B remain completely unaware that their "secure" channel has been compromised, since the DH exchange completes successfully on both ends.

This is particularly dangerous in the given scenario because the network is explicitly stated to be vulnerable to attacks, meaning an active adversary capable of intercepting and modifying traffic is a realistic threat, not just a passive eavesdropper.

Mitigation: Add Authentication to the Key Exchange

Several approaches can address this vulnerability:

1. Digital Signatures with PKI: Each party signs their DH public value (A or B) using their own long-term private signing key. The receiving party verifies this signature using the sender's public key, which is itself certified by a trusted Certificate Authority (CA). This ensures that even if Eve intercepts the traffic, she cannot forge a valid signature for a fraudulent public value, so the MITM substitution would be detected and rejected.
2. Station-to-Station (STS) Protocol: An authenticated variant of Diffie-Hellman that explicitly incorporates digital signatures and a challenge-response mechanism into the exchange itself, so authentication and key agreement happen together rather than as an add-on.
3. TLS with Ephemeral Diffie-Hellman (DHE/ECDHE): In practice, this is how the problem is solved on the modern internet — TLS uses DH (or ECDH) for the actual key exchange but wraps it inside a protocol where the server (and optionally the client) presents an X.509 certificate, cryptographically verified via a certificate chain rooted in a trusted CA, before the key exchange values are trusted.
4. Pre-shared authentication material: In smaller, closed environments, the two parties could pre-share a secret (like a password or key fingerprint, verified out-of-band, e.g., by phone) to authenticate the

DH exchange (as in protocols like SPEKE or PAKE variants).

Conclusion: The mathematics of Diffie-Hellman successfully protects the shared secret from passive eavesdroppers, but without an accompanying authentication mechanism, it remains fully vulnerable to active Man-in-the-Middle attacks on an unsecured network. Binding the exchange to verified identities via digital signatures and PKI is the standard and necessary solution.

4. Public Key Cryptography for Email Security (RSA)

A. How RSA can be used for encryption and decryption

RSA is an asymmetric (public-key) cryptographic algorithm, meaning it uses a mathematically linked pair of keys — a public key (which can be shared openly) and a private key (which must be kept secret by its owner). Its security is based on the computational difficulty of factoring the product of two very large prime numbers.

Key generation (performed once per user):

1. Choose two large distinct prime numbers, p and q .
2. Compute $n = p \times q$ (this becomes the modulus, part of both keys).
3. Compute Euler's totient: $\phi(n) = (p-1)(q-1)$.
4. Choose a public exponent e , such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$ (commonly $e = 65537$).
5. Compute the private exponent d , the modular multiplicative inverse of $e \bmod \phi(n)$, such that $e \cdot d \equiv 1 \pmod{\phi(n)}$.
6. The public key = (e, n) is published/shared; the private key = (d, n) is kept secret.

Applying this to secure email communication:

1. Key distribution: Each employee/user in the organization generates (or is issued) an RSA key pair. Public keys are published — often through a company directory or embedded in digital certificates — while private keys remain solely on each user's device or secure token.
2. Sending a secure email (Encryption): When User A wants to send a

confidential email to User B, User A obtains User B's public key. Because RSA is computationally expensive for large data, in practice a hybrid encryption scheme is used:

- A random symmetric session key (e.g., for AES) is generated.
 - The actual email content is encrypted with this fast symmetric AES key.
 - The AES session key itself is then encrypted using User B's RSA public key: $\text{EncryptedKey} = (\text{SessionKey})^e \bmod n$.
 - Both the AES-encrypted email and the RSA-encrypted session key are sent to User B.
3. Receiving and decrypting the email (Decryption): User B uses their own RSA private key to decrypt the session key: $\text{SessionKey} = (\text{EncryptedKey})^d \bmod n$. Since only User B possesses the corresponding private key, only User B can recover the session key – and therefore only User B can decrypt and read the actual email content.
4. Digital signatures for authenticity (often used alongside encryption): User A can also compute a hash of the email and sign it with their own private key. User B then verifies this signature using User A's public key, confirming that the email genuinely originated from User A and was not altered in transit. This provides authentication and non-repudiation, on top of the confidentiality RSA/AES provide.

This combination (RSA for key exchange/signing + AES for bulk content encryption) is essentially the model used by real-world secure email standards such as PGP/GPG and S/MIME.

B. Role of key distribution and management in maintaining security

The mathematical strength of RSA is only as good as the surrounding key management practices – a common saying in cryptography is that "cryptography is rarely broken, it's usually bypassed" through poor key handling. Several factors are critical:

1. Authenticity of public keys (Public Key Infrastructure – PKI): The biggest practical risk is not RSA's math being broken, but a user unknowingly encrypting a message using an *attacker's* public key instead of the intended recipient's – a form of impersonation.

Organizations mitigate this using:

- Digital certificates, issued by a trusted Certificate Authority (CA), which cryptographically bind a public key to a verified identity (e.g., an employee's name and email address).
 - A Certificate Revocation List (CRL) or OCSP (Online Certificate Status Protocol) to check whether a certificate has been revoked (e.g., due to key compromise or an employee leaving the organization) before trusting it.
2. Secure private key storage: Since the entire security of a user's encrypted communications depends on their private key remaining secret, organizations must ensure private keys are stored in hardware security modules (HSMs), encrypted key stores, or smart cards/USB tokens — never in plaintext files or emails.
 3. Key size and algorithm currency: As computing power grows, RSA key sizes must be periodically increased (the current recommended minimum is 2048 bits, with 3072/4096 bits for higher long-term security) to remain resistant to factoring attacks. Organizations must have policies to phase out weak/legacy key sizes.
 4. Key rotation and expiration: Keys should not be used indefinitely. Regular key rotation limits the amount of data exposed if a private key is ever compromised, and certificates are typically issued with defined expiration dates to force periodic renewal and re-verification of identity.
 5. Secure key distribution channels: When public keys or certificates are first distributed (e.g., onboarding a new employee), this must happen through a verified, tamper-proof channel to prevent a "key substitution" attack at the outset.
 6. Revocation handling: If an employee's private key is lost, stolen, or the employee leaves the organization, there must be a clear, fast process to revoke that key pair and issue a new one, updating all relevant directories/certificates so others stop trusting the old public key.

Conclusion: RSA provides the mathematical foundation for confidentiality and authenticity in the organization's email system, but its real-world security depends heavily on rigorous key distribution and lifecycle management — without a trustworthy PKI, secure private key storage, and clear revocation procedures, the cryptographic strength of

RSA itself can be completely undermined by human or procedural failures.

5. Cryptography for IoT Devices

A. Constraints of IoT devices

Smart home IoT devices (such as smart bulbs, door locks, thermostats, sensors, and cameras) differ fundamentally from traditional computing devices (servers, PCs, smartphones) in ways that directly impact which cryptographic algorithms are practical to deploy:

1. Limited processing power: IoT devices typically use low-cost, low-power microcontrollers (MCUs) with far less computational capability than general-purpose CPUs. Performing computationally intensive operations — such as RSA's large-integer modular exponentiation with 2048-bit or larger keys — can take a noticeably long time and consume disproportionate processing resources on such hardware.
2. Limited memory (RAM/ROM/Flash): Many IoT devices have only kilobytes of RAM and limited flash storage, which restricts both the size of cryptographic keys that can be stored/handled and the complexity of the cryptographic code/libraries that can be embedded in firmware.
3. Battery and energy constraints: A large portion of smart home devices run on battery power or are designed for minimal energy draw (for cost and heat reasons). Every cryptographic operation consumes energy, so computationally heavier algorithms directly reduce battery life — a critical usability factor for consumer smart home products.
4. Bandwidth and communication constraints: IoT devices often communicate over low-power, low-bandwidth wireless protocols (e.g., Zigbee, Bluetooth Low Energy, LoRa). Larger keys and larger ciphertext/signature sizes (as required by RSA) increase the amount of data that must be transmitted, consuming more bandwidth and more energy per transmission.
5. Scalability across many devices: A smart home system may include dozens of connected devices, each requiring secure key generation, exchange, and management. An approach that is

computationally/storage-expensive per device becomes impractical to scale across an entire home (or, at larger scale, an entire smart city) deployment.

6. Cost constraints: IoT devices are typically produced at very low cost per unit; adding powerful cryptographic co-processors purely to support computationally heavy algorithms like RSA would increase the bill-of-materials cost significantly.

B. Recommended cryptographic approach and justification

Recommendation: Elliptic Curve Cryptography (ECC), used via ECDH (Elliptic Curve Diffie-Hellman) for key exchange and ECDSA (Elliptic Curve Digital Signature Algorithm) for authentication, rather than RSA.

Why ECC is better suited to these constraints:

1. Smaller key sizes for equivalent security: ECC's security is based on the elliptic curve discrete logarithm problem, which is significantly harder to solve (relative to key size) than RSA's integer factorization problem. As a result, ECC achieves comparable security strength to RSA with much smaller keys – for example:
 - A 256-bit ECC key offers security roughly equivalent to a 3072-bit RSA key.
 - A 160-bit ECC key is roughly equivalent to a 1024-bit RSA key.

This dramatic reduction in key size directly benefits every constrained resource on an IoT device.

1. Lower computational overhead: Because ECC operations work with much smaller numbers than RSA (for equivalent security), the elliptic curve point multiplication operations at the core of ECC require significantly fewer CPU cycles than RSA's large modular exponentiations. This translates to faster cryptographic operations, which matters both for responsiveness (e.g., a smart lock authenticating quickly) and for battery life.
2. Reduced memory footprint: Smaller keys and smaller intermediate computation values mean ECC implementations require less RAM and flash storage – a critical advantage for MCUs with only tens of kilobytes of memory available.
3. Lower bandwidth consumption: Smaller keys, smaller signatures,

and smaller ciphertext mean less data needs to be transmitted over the constrained wireless links used in home automation protocols, further reducing energy consumption per operation (since radio transmission is often one of the most energy-expensive operations an IoT device performs).

4. Better scalability: Because each device's cryptographic footprint (in terms of computation, storage, and bandwidth) is smaller, ECC scales more efficiently across a home (or larger) full of many simultaneously-connected devices, compared to RSA.
5. Industry alignment: ECC is already the basis for many modern lightweight/IoT-focused security standards and protocols (e.g., ECDHE in TLS 1.3, which is now widely used even in constrained-device contexts, and various IoT security frameworks that explicitly recommend ECC over RSA for exactly these reasons).

Conclusion: Given the smart home system's constraints — limited processing power, memory, battery life, and bandwidth — ECC is the clearly superior cryptographic approach compared to RSA. It delivers equivalent (or stronger) security guarantees while consuming a fraction of the computational, memory, and energy resources that RSA would require, making it the practical and justified choice for securing communication among low-power IoT devices in this scenario.